



UNIT 04: CONSTRUCTIONS SHORS

CODE

Table of Contents

INTRODUCTION.....	3
LEARNING OBJECTIVES.....	4
4.1 SHOR'S CODE: DEFINITION	4
4.1.1 HISTORICAL CONTEXT	4
4.1.2 THE CONCATENATION PRINCIPLE	5
4.1.3 CODE PARAMETERS $[[9,1,3]]$	8
4.1.4 LOGICAL CODEWORDS	10
4.2 ENCODING AND DECODING	12
4.2.1 ENCODING CIRCUIT OVERVIEW.....	12
4.2.2 STAGE 1: PHASE-FLIP ENCODING	13
4.2.3 STAGE 2: BIT-FLIP ENCODING	15
4.2.4 STABILIZER GENERATORS	16
4.3 ERROR CORRECTION.....	19
4.3.1 SYNDROME MEASUREMENT PROTOCOL	19
4.3.2 BIT-FLIP ERROR DETECTION AND CORRECTION	21
4.3.3 PHASE-FLIP ERROR DETECTION AND CORRECTION	22
4.3.4 COMBINED ERRORS AND ERROR DISCRETIZATION	23



4.4 SUMMARY	27
4.5 MIND MAP	28
4.6 GLOSSARY	30
4.7 SELF-ASSESSMENT QUESTIONS	30
4.8 TERMINAL QUESTIONS	34
4.8.1 SHORT ANSWER QUESTIONS	34
4.8.2 LONG ANSWER QUESTIONS	35
4.9 ANSWERS	35
4.9.1 SELF-ASSESSMENT ANSWERS	35
4.9.2 TERMINAL QUESTION ANSWERS	36
4.10 REFERENCES	38



INTRODUCTION

In 1995, the field of quantum computing faced an existential crisis. Shor's factoring algorithm had demonstrated the extraordinary potential of quantum computers, but a fundamental problem loomed: quantum systems are extraordinarily fragile. The slightest interaction with the environment causes decoherence, scrambling quantum information in ways that seemed impossible to reverse. Many physicists believed this fragility made large-scale quantum computation fundamentally impossible.

Then Peter Shor proposed something remarkable — a nine-qubit quantum code that could protect against *any* single-qubit error. This wasn't just an incremental improvement; it was a conceptual breakthrough that proved quantum error correction was possible despite the no-cloning theorem. The sceptics had argued that since quantum states cannot be copied, the redundancy-based techniques of classical error correction could not work. Shor showed they were wrong.

The key insight was elegantly simple: combine two codes that each handle one type of error. The three-qubit bit-flip code protects against X errors, and the three-qubit phase-flip code protects against Z errors. Neither code alone can handle both error types. But by *nesting* one code inside the other — a technique called concatenation — Shor created a code that protects against arbitrary single-qubit errors, including the **continuous superposition of Pauli errors** that lie between pure bit-flips and pure phase-flips.

This unit examines Shor's $[[9,1,3]]$ code in detail. We begin with its definition and the concatenation principle that makes it work, then explore the encoding circuit and stabilizer structure that define its properties, and finally understand how syndrome measurement enables error correction. Along the way, we encounter one of quantum mechanics' most surprising features: the discretization of continuous errors through measurement. This principle underlies all quantum error correction and represents one of the field's deepest insights.



LEARNING OBJECTIVES

By the end of this unit, you will be able to:

1. Explain how concatenating the bit-flip and phase-flip codes protects against arbitrary single-qubit errors.
2. Construct the logical codewords of Shor's $[[9,1,3]]$ code and verify their normalisation.
3. Apply the two-stage encoding circuit to trace quantum information through each encoding stage.
4. Analyse the 8-bit syndrome table to identify errors and determine recovery operations.
5. Evaluate why correcting discrete Pauli errors suffices to correct arbitrary continuous errors.

4.1 SHOR'S CODE: DEFINITION

4.1.1 HISTORICAL CONTEXT

The year 1994 marked a turning point for quantum computing. Peter Shor's factoring algorithm demonstrated that a quantum computer could solve problems believed intractable for classical machines — specifically, factoring large integers in polynomial time. This result immediately attracted attention from cryptographers, governments, and physicists worldwide, since the security of RSA encryption depends on the difficulty of factoring.

However, this excitement was tempered by a sobering reality. Quantum states are extraordinarily sensitive to environmental disturbances. A single stray photon, a slight temperature fluctuation, or an unwanted electromagnetic field can cause *decoherence* — the gradual loss of quantum coherence that destroys the delicate superpositions on which quantum algorithms depend. While a classical bit can be protected by making it physically



robust, a quantum superposition $\alpha|0\rangle + \beta|1\rangle$ exists in a fundamentally more delicate state.

Many physicists argued that this fragility made quantum computation fundamentally impossible at scale. The no-cloning theorem seemed to rule out the redundancy-based approaches used in classical error correction. Classical codes work by making multiple copies of information — if you have three copies and one is corrupted, majority voting recovers the original. But quantum mechanics forbids copying unknown quantum states. If you cannot copy quantum information, how can you protect it?

In 1995, Shor answered this challenge with a nine-qubit code that could correct arbitrary single-qubit errors. Nielsen and Chuang highlight that this simple quantum code can protect against the effects of any arbitrary error on a single qubit — a result many had believed impossible.

This breakthrough demonstrated that quantum error correction was possible despite the no-cloning theorem. The key insight was encoding information in *entangled* states rather than copying it. The encoded state is not three copies of the original — it is a highly entangled nine-qubit state where the original information is spread across all qubits in a way that can survive local errors.

Shor's code opened the door to fault-tolerant quantum computation and launched an entire subfield of quantum information science. The timeline that followed was rapid: in 1996, Andrew Steane proposed a seven-qubit code with elegant symmetry properties that is an important example of CSS (Calderbank-Shor-Steane) codes, which were independently developed by Calderbank-Shor and Steane that same year. Also in 1996, the five-qubit code was discovered — the smallest possible code that can correct arbitrary single-qubit errors, achieving the quantum Hamming bound.

4.1.2 THE CONCATENATION PRINCIPLE

Shor's code is built from two simpler codes that we encountered in Unit 3: the three-qubit bit-flip code and the three-qubit phase-flip code. Understanding why neither code alone



suffices — and how combining them solves the problem — is key to understanding Shor's construction.

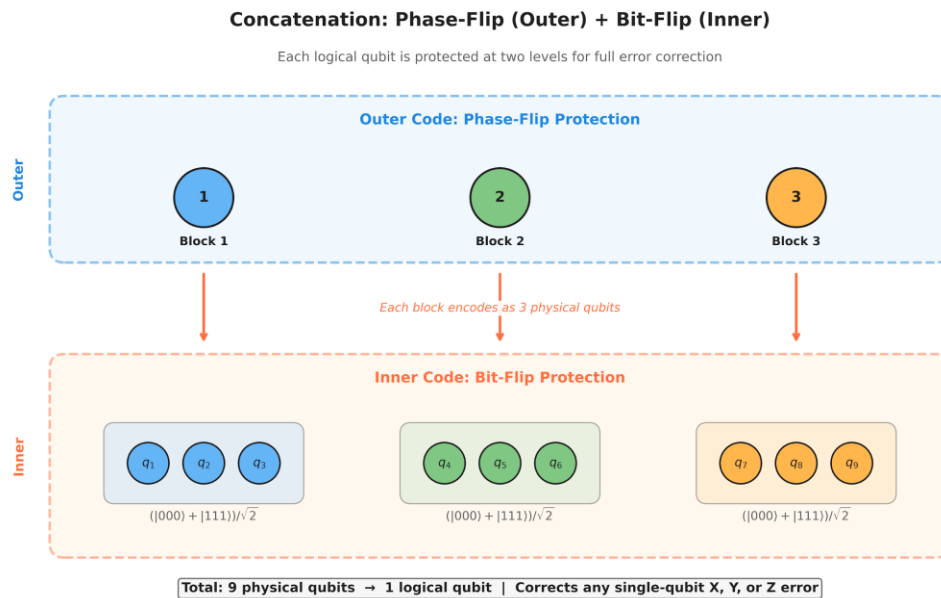


Figure 4.1: The Concatenation Principle — the Outer Phase-flip Code Creates Three Logical Blocks, and the Inner Bit-flip Code Expands Each Block to Three Physical Qubits.

The **bit-flip code** encodes:

- $|0\rangle \rightarrow |000\rangle$
- $|1\rangle \rightarrow |111\rangle$

This code can detect and correct single X (bit-flip) errors by majority voting. If we measure the three qubits and find $|001\rangle$, we know the third qubit was flipped and can correct it. However, this code is completely blind to Z (phase-flip) errors. To see why, note that $Z|0\rangle = |0\rangle$ and $Z|1\rangle = -|1\rangle$. Applying Z to any qubit of $|000\rangle$ gives $|000\rangle$ (unchanged), and applying Z to any qubit of $|111\rangle$ gives $-|111\rangle$. The state $\alpha|000\rangle + \beta|111\rangle$ becomes $\alpha|000\rangle - \beta|111\rangle$ — the phase information is corrupted, but the bit-flip code cannot detect this because both $|000\rangle$ and $|111\rangle$ remain valid bit strings.

The **phase-flip code** encodes in the Hadamard-rotated basis:



UNIT 04: CONSTRUCTIONS SHORS CODE

- $|0\rangle \rightarrow |+++ \rangle = (|0\rangle + |1\rangle)^{\otimes 3} / (2\sqrt{2})$
- $|1\rangle \rightarrow |-- \rangle = (|0\rangle - |1\rangle)^{\otimes 3} / (2\sqrt{2})$

This code can detect and correct single Z errors, which flip $|+\rangle$ to $|-\rangle$ and vice versa. However, it cannot detect X errors, because $X|+\rangle = |+\rangle$ and $X|-\rangle = |-\rangle$ — bit-flips leave the $\pm$ states unchanged.

Neither code alone can protect against both error types. As Nielsen and Chuang explain, Shor's code is fundamentally a combination of the three-qubit phase-flip and bit-flip codes.

The solution is **concatenation** — nesting one code inside another. This is not the same as running two codes in parallel or in sequence; it is a hierarchical structure where one code's outputs become another code's inputs. In Shor's construction:

1. **Outer code (phase-flip):** The logical qubit is first encoded using the phase-flip code, creating three "block" qubits in states $|+\rangle$ or $|-\rangle$
2. **Inner code (bit-flip):** Each of these three block qubits is then encoded using the bit-flip code, expanding $|+\rangle$ to $(|000\rangle + |111\rangle)/\sqrt{2}$ and $|-\rangle$ to $(|000\rangle - |111\rangle)/\sqrt{2}$

This creates a two-level protection structure:

- **Between blocks:** The phase-flip code protects against Z errors that affect entire blocks
- **Within blocks:** The bit-flip code protects against X errors on individual qubits

Nielsen and Chuang note: "This method of encoding using a hierarchy of levels is known as *concatenation*. It's a great trick for obtaining new codes from old."

The key insight is that these two protection mechanisms operate at different "scales" — the bit-flip code works within each three-qubit block, while the phase-flip code works across the three blocks. An X error on one qubit is caught by the bit-flip code. A Z error on one qubit affects the sign of its block's state, which is caught by the phase-flip code. The two detection mechanisms don't interfere with each other because they look at different aspects of the encoded state.



***Common Misconception:** Students often think Shor's code is complex because it uses nine qubits. In reality, it is simply two three-qubit codes combined through nesting — one of the most elegant constructions in quantum error correction. Once you understand the bit-flip and phase-flip codes, Shor's code follows naturally.*

4.1.3 CODE PARAMETERS $[[9,1,3]]$

Shor's code has parameters $[[9,1,3]]$ in the standard notation introduced in Unit 3. Let us examine what each parameter means and why these specific values arise from the concatenation construction.

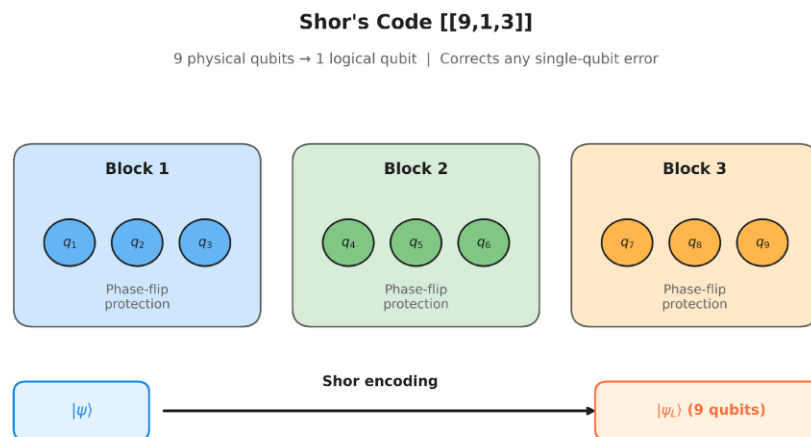


Figure 4.2: The Block Structure of Shor's $[[9,1,3]]$ Code — Nine Physical Qubits Arranged in Three Colour-coded Blocks, Each Protected by the Bit-flip Code.

n = 9: The code uses nine physical qubits, arranged as three blocks of three qubits each:

- Block 1: Qubits 1, 2, 3
- Block 2: Qubits 4, 5, 6
- Block 3: Qubits 7, 8, 9



UNIT 04: CONSTRUCTIONS SHORS CODE

This structure follows directly from concatenation: the outer phase-flip code creates three logical qubits, and the inner bit-flip code expands each of these to three physical qubits, giving $3 \times 3 = 9$ total.

k = 1: The code encodes one logical qubit. The two-dimensional code space is spanned by the logical states $|\bar{0}\rangle$ and $|\bar{1}\rangle$. Any logical state $\alpha|\bar{0}\rangle + \beta|\bar{1}\rangle$ can be encoded, where α and β are arbitrary complex amplitudes satisfying $|\alpha|^2 + |\beta|^2 = 1$.

d = 3: The code distance is 3, meaning the minimum weight of a non-trivial logical operator is 3. This implies the code can correct any single-qubit error (since correcting t errors requires $d \geq 2t + 1$) and detect (but not uniquely correct) two-qubit errors.

The relationship between these parameters follows the stabilizer formalism: with $n = 9$ physical qubits and 8 independent stabilizer generators, the code space has dimension $2^{9-8} = 2$, encoding $k = 1$ logical qubit. The formula $n - k = 8$ tells us there are 8 constraints (stabilizer generators) on the 9-qubit Hilbert space.

Code	n	k	d	Error Correction Capability
3-qubit bit-flip	3	1	1*	Single X error only
3-qubit phase-flip	3	1	1*	Single Z error only
Shor $[[9,1,3]]$	9	1	3	Any single-qubit error

Table 4.1: Comparison of Quantum Code Parameters — the Shor Code Combines Bit-flip and Phase-flip Protection to Achieve Distance 3.

*Note: The distance $d=1$ for these codes refers to the full quantum code distance, reflecting their vulnerability to the complementary error type (Z errors for bit-flip, X errors



for phase-flip). Within their respective error models, each code can correct one error of its type.

The cost of this expanded protection is significant: we use nine physical qubits to encode one logical qubit, a ninefold overhead. This ratio, sometimes called the encoding rate $k/n = 1/9 \approx 0.11$, seems inefficient. Later codes achieve better rates — the five-qubit code has rate $1/5 = 0.20$, and CSS codes offer various trade-offs. However, Shor's code has pedagogical value and historical importance as the first complete quantum error-correcting code.

4.1.4 LOGICAL CODEWORDS

The logical basis states for Shor's code are elegant mathematical objects that encode quantum information across nine entangled qubits. Understanding their structure is essential for working with the code.

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)(|000\rangle + |111\rangle)}{2\sqrt{2}}$$

$$|1_L\rangle = \frac{(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)(|000\rangle - |111\rangle)}{2\sqrt{2}}$$

How to Read: "Zero-logical equals the tensor product of three identical $(|000\rangle + |111\rangle)$ blocks, all divided by $2\sqrt{2}$ for normalization. One-logical has minus signs in each block instead of plus signs."

Where:

- $|0_L\rangle, |1_L\rangle$ are the logical basis states (the encoded computational basis)
- $(|000\rangle + |111\rangle)$ is the "+"-state of one three-qubit block, sometimes written $|GHZ_+\rangle$
- $(|000\rangle - |111\rangle)$ is the "-"-state of one three-qubit block, sometimes written $|GHZ_-\rangle$
- $2\sqrt{2}$ is the normalization factor: each block contributes $\sqrt{2}$ from $(|000\rangle \pm |111\rangle)/\sqrt{2}$, and three blocks give $(\sqrt{2})^3 = 2\sqrt{2}$

**Block Structure:**

Block	Qubits	State in	0 _L	State in	1 _L
1	1, 2, 3	(	000) ⁺	111) ⁺ /√2	(
2	4, 5, 6	(	000) ⁺	111) ⁺ /√2	(
3	7, 8, 9	(	000) ⁺	111) ⁺ /√2	(

Table 4.2: Block Structure of Shor's Code — Each Block Encodes the Logical Basis State Using Three-qubit GHZ-like States.

Notice that $|0_L\rangle$ and $|1_L\rangle$ differ only in the relative signs within each block. This sign difference — plus versus minus — is the phase information that distinguishes the two logical states. The phase-flip code at the outer level protects this sign information.

Expanding $|0_L\rangle$ explicitly as a sum of computational basis states, we get eight terms by choosing either $|000\rangle$ or $|111\rangle$ from each block:

$$|0_L\rangle = \frac{1}{2\sqrt{2}} (|000000000\rangle + |000000111\rangle + |000111000\rangle + |000111111\rangle \\ + |111000000\rangle + |111000111\rangle + |111111000\rangle + |111111111\rangle)$$

This expansion contains 8 terms (2^3 combinations from choosing $|000\rangle$ or $|111\rangle$ in each of the three blocks). Each term has amplitude $1/(2\sqrt{2})$, and we can verify normalization: 8 terms, each with probability $|1/(2\sqrt{2})|^2 = 1/8$, sum to total probability $8 \times (1/8) = 1$. ✓

For $|1_L\rangle$, the expansion is similar but with signs determined by parity:

$$|1_L\rangle = \frac{1}{2\sqrt{2}} (|000000000\rangle - |000000111\rangle - |000111000\rangle + |000111111\rangle \\ - |111000000\rangle + |111000111\rangle + |111111000\rangle - |111111111\rangle)$$

The sign pattern follows a rule: the coefficient is positive when an even number of blocks are in the $|111\rangle$ state, and negative when an odd number are. This can be understood from the fact that each $|111\rangle$ block in $|1_L\rangle$ contributes a minus sign from the $(|000\rangle - |111\rangle)$ factor.



Activity: Write out $|1\rangle$ expanded as a sum of 8 computational basis states. Verify that the signs follow the pattern: positive when an even number of blocks are in the $|111\rangle$ state, negative when an odd number are. Count the number of positive and negative terms.

4.2 ENCODING AND DECODING

4.2.1 ENCODING CIRCUIT OVERVIEW

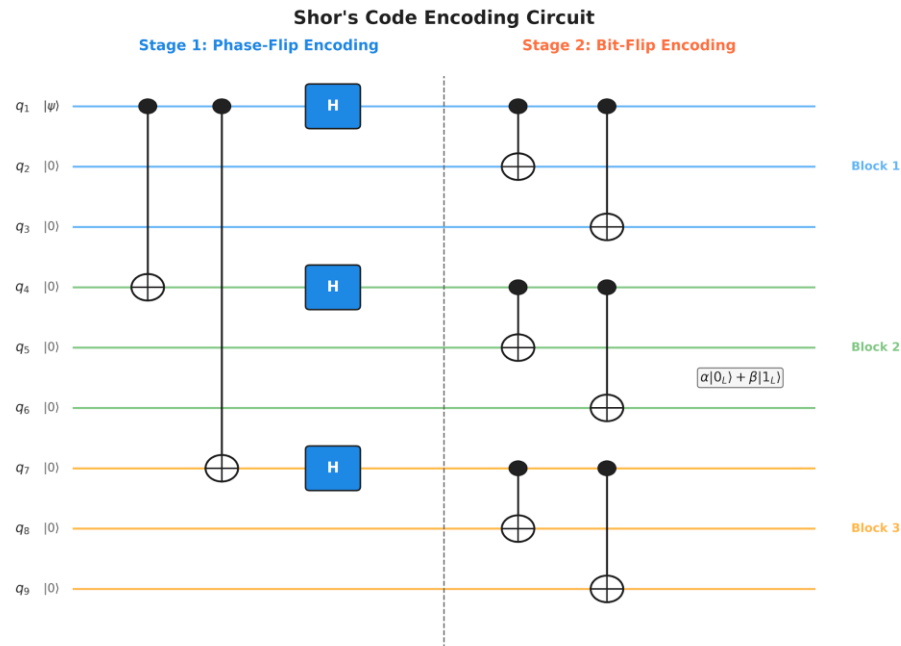


Figure 4.3: The Complete Encoding Circuit for Shor's Code — Stage 1 (phase-flip Encoding)

Uses 2 CNOTs and 3 Hadamards; Stage 2 (bit-flip Encoding) Uses 6 CNOTs.

The encoding circuit transforms an arbitrary single-qubit state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ into the encoded state $\alpha|0_L\rangle + \beta|1_L\rangle$. This transformation must be unitary (reversible) and must work for any input state — it cannot depend on knowing α and β in advance.

The input to the encoding circuit consists of:



UNIT 04: CONSTRUCTIONS SHORS CODE

- **Qubit 1:** The data qubit in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$
- **Qubits 2-9:** Eight ancilla qubits, each initialized to $|0\rangle$

The output is:

- **Qubits 1-9:** The nine-qubit encoded state $\alpha|0_L\rangle + \beta|1_L\rangle$

The circuit consists of 11 gates arranged in two stages that mirror the concatenation structure:

Stage	Gates	Purpose
Stage 1	2 CNOT + 3 Hadamard	Phase-flip encoding
Stage 2	6 CNOT	Bit-flip encoding

Table 4.3: Two-stage Encoding Process for Shor's Code, Showing the Gate Count and Purpose of Each Stage.

The total gate count is 8 CNOT gates and 3 Hadamard gates. This relatively simple circuit reflects the elegant structure of concatenated codes — the complexity of the code's error-correcting properties emerges from the structure, not from circuit depth.

The circuit can be viewed as first preparing the phase-flip encoding (creating three entangled blocks) and then expanding each block using bit-flip encoding. Understanding this two-stage structure is key to understanding how the code achieves protection against both error types.

4.2.2 STAGE 1: PHASE-FLIP ENCODING

Stage 1 creates the three-block structure that protects against phase-flip errors. This stage operates only on qubits 1, 4, and 7 — the "block leaders" that will become the control qubits for the subsequent bit-flip encoding.

Step 1: Create entanglement across blocks

Apply CNOT from qubit 1 to qubit 4, and CNOT from qubit 1 to qubit 7. The CNOT gate flips the target qubit if and only if the control qubit is $|1\rangle$.



UNIT 04: CONSTRUCTIONS SHORS CODE

Starting from $|\psi, 0, 0\rangle$ on qubits 1, 4, 7:

- If qubit 1 is $|0\rangle$: both CNOTs do nothing, result is $|000\rangle$
- If qubit 1 is $|1\rangle$: both CNOTs flip their targets, result is $|111\rangle$

For a superposition input $\alpha|0\rangle + \beta|1\rangle$, the output is $\alpha|000\rangle + \beta|111\rangle$. This is a GHZ-like entangled state across the three block leaders.

Step 2: Convert to X-basis

Apply Hadamard gates to qubits 1, 4, and 7. The Hadamard gate transforms between the Z-basis and X-basis:

- $H|0\rangle = |+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$
- $H|1\rangle = |-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$

Applying $H \otimes H \otimes H$ to our entangled state:

- $|000\rangle \rightarrow |+++ \rangle$ (tensor product of three $|+\rangle$ states)
- $|111\rangle \rightarrow |--- \rangle$ (tensor product of three $|-\rangle$ states)

After Stage 1, our state has transformed:

- $\alpha|0\rangle + \beta|1\rangle \rightarrow \alpha|+++ \rangle + \beta|--- \rangle$ (on qubits 1, 4, 7)

This is exactly the phase-flip encoding! The logical $|0\rangle$ maps to $|+++ \rangle$ and logical $|1\rangle$ maps to $|--- \rangle$, which is the standard phase-flip code construction.

Gate-by-gate trace for an arbitrary input:

Step	Operation	State (qubits 1, 4, 7)
Initial	—	α
After $\text{CNOT}_{1 \rightarrow 4}$	CNOT	α
After $\text{CNOT}_{1 \rightarrow 7}$	CNOT	α
After H_1	Hadamard	α
After H_4	Hadamard	(expanding)
After H_7	Hadamard	α

Table 4.4: Gate-by-gate Trace of Stage 1 (phase-flip) Encoding, Showing the State Evolution of Qubits 1, 4, and 7.



Think About It: Why do we apply the Hadamard gates after the CNOTs, not before? Consider what state we need before applying bit-flip encoding to each block.

The answer lies in the structure of the bit-flip code: it expects a Z-basis input ($|0\rangle$ or $|1\rangle$), or more precisely, a state in the X-basis ($|+\rangle$ or $|-\rangle$) that will be expanded to $(|000\rangle+|111\rangle)/\sqrt{2}$ or $(|000\rangle-|111\rangle)/\sqrt{2}$. The CNOTs create a GHZ-like entangled state in the Z-basis, establishing correlations between the three block leaders. Then the Hadamards convert each block leader to the X-basis, preparing them for bit-flip encoding. If we applied Hadamards first, the subsequent CNOTs would create different (and incorrect) entanglement patterns.

4.2.3 STAGE 2: BIT-FLIP ENCODING

Stage 2 expands each of the three block leaders (qubits 1, 4, 7) into a three-qubit bit-flip encoded block. This stage creates the redundancy needed to protect against bit-flip errors within each block.

Block 1 (qubits 1, 2, 3):

- CNOT from qubit 1 to qubit 2
- CNOT from qubit 1 to qubit 3
- Input state $|+\rangle$ on qubit 1, $|00\rangle$ on qubits 2,3
- Output: $|+\rangle = (|0\rangle+|1\rangle)/\sqrt{2} \rightarrow (|000\rangle+|111\rangle)/\sqrt{2}$

Similarly for $|-\rangle$:

- $|-\rangle = (|0\rangle-|1\rangle)/\sqrt{2} \rightarrow (|000\rangle-|111\rangle)/\sqrt{2}$

Block 2 (qubits 4, 5, 6): Same operations with qubit 4 as control, qubits 5 and 6 as targets.

Block 3 (qubits 7, 8, 9): Same operations with qubit 7 as control, qubits 8 and 9 as targets.

Note that all six CNOTs in Stage 2 can be applied simultaneously (in parallel), since they operate on different qubits. In a physical implementation, this parallelism reduces circuit depth.

Complete transformation:



Starting from $\alpha|+++ \rangle + \beta|--- \rangle$ on block leaders (after Stage 1), we expand each $|\pm \rangle$ to its three-qubit form:

$$\begin{aligned} & \alpha|+++ \rangle + \beta|--- \rangle \\ \rightarrow & \alpha \frac{(|000 \rangle + |111 \rangle)^{\otimes 3}}{(\sqrt{2})^3} + \beta \frac{(|000 \rangle - |111 \rangle)^{\otimes 3}}{(\sqrt{2})^3} \\ & = \alpha|0_L \rangle + \beta|1_L \rangle \end{aligned}$$

This completes the encoding. The original qubit $|\psi \rangle = \alpha|0 \rangle + \beta|1 \rangle$ has been transformed into the nine-qubit encoded state $\alpha|0_L \rangle + \beta|1_L \rangle$.

Complete circuit summary:

Order	Gate	Control→Target	Purpose
1	CNOT	1→4	Entangle block leaders
2	CNOT	1→7	Entangle block leaders
3	H	1	Convert to X-basis
4	H	4	Convert to X-basis
5	H	7	Convert to X-basis
6	CNOT	1→2	Bit-flip encode block 1
7	CNOT	1→3	Bit-flip encode block 1
8	CNOT	4→5	Bit-flip encode block 2
9	CNOT	4→6	Bit-flip encode block 2
10	CNOT	7→8	Bit-flip encode block 3
11	CNOT	7→9	Bit-flip encode block 3



Table 4.5: Complete Gate Sequence for the Shor Code Encoding Circuit — 11 Gates Applied in Order with Control-target Connections.

4.2.4 STABILIZER GENERATORS

The stabilizer group of Shor's code is generated by eight independent Pauli operators. These generators define the code space (as their +1 eigenspace) and enable error detection through syndrome measurement. Understanding the stabilizer structure reveals how the code detects different types of errors.

Bit-flip detection generators (Z-type):

Generator	Operator	Full Form	Function
M_1	Z_1Z_2	ZZIIIIII	Detects bit-flips in block 1
M_2	Z_2Z_3	IZZIIIIII	Detects bit-flips in block 1
M_3	Z_4Z_5	IIIZZIII	Detects bit-flips in block 2
M_4	Z_5Z_6	IIIZZIII	Detects bit-flips in block 2
M_5	Z_7Z_8	IIIIIZZI	Detects bit-flips in block 3
M_6	Z_8Z_9	IIIIIZZI	Detects bit-flips in block 3

Table 4.6: Z-type Stabilizer Generators (M_1 – M_6) for the Shor Code — Each Measures Parity between Adjacent Qubits Within a Block to Detect Bit-flip Errors.

Phase-flip detection generators (X-type):

Generator	Operator	Full Form	Function
-----------	----------	-----------	----------



M_7	$X_1X_2X_3X_4X_5X_6$	XXXXXXIII	Compares phase of blocks 1 and 2
M_8	$X_4X_5X_6X_7X_8X_9$	IIIXXXXXX	Compares phase of blocks 2 and 3

Table 4.7: X-type Stabilizer Generators (M_7 – M_8) for the Shor Code — Each Measures the Relative Phase between Blocks to Detect Phase-flip Errors.

As Gottesman notes in his thesis, the two valid codewords $|0\rangle$ and $|1\rangle$ in Shor's code are eigenvectors of all eight generators with eigenvalue +1.

Verifying the stabilizer property:

To verify that $|0L\rangle$ is a +1 eigenstate of $M_1 = Z_1Z_2$, we apply Z_1Z_2 to each term in $|0L\rangle$:

- $Z_1Z_2|000\dots\rangle = (+1)(+1)|000\dots\rangle = |000\dots\rangle$
- $Z_1Z_2|111\dots\rangle = (-1)(-1)|111\dots\rangle = |111\dots\rangle$

Since both basis states within each block are eigenstates with eigenvalue +1, the full codeword $|0L\rangle$ is also a +1 eigenstate. The same analysis applies to $|1L\rangle$ and to all other generators.

Stabilizer properties:

1. **Commutativity:** All generators commute: $[M_i, M_j] = 0$ for all pairs i, j . This is required for simultaneous measurement.
2. **Eigenvalue condition:** $M_i|0L\rangle = +|0L\rangle$ and $M_i|1L\rangle = +|1L\rangle$ for all $i = 1, \dots, 8$. This defines the code space.
3. **Independence:** No generator can be expressed as a product of the others. This ensures we have exactly 8 independent constraints.
4. **Dimension count:** 8 generators for 9 qubits gives code dimension $2^{9-8} = 2$, confirming $k = 1$.

How to read the generator notation: Each generator is a tensor product of Pauli operators across all nine qubits. For $M_1 = Z_1Z_2$, the full operator is $Z \otimes Z \otimes I \otimes I \otimes I \otimes I \otimes I \otimes I \otimes I$, $\equiv Z \otimes Z \otimes I \otimes I \otimes I \otimes I \otimes I \otimes I$



meaning we apply Z to qubits 1 and 2, and the identity I to all other qubits. When we write Z_1Z_2 , we implicitly mean the identity acts on unlisted positions.

***Did You Know?** The stabilizer formalism introduced in Unit 6 generalizes this generator structure and provides an efficient framework for describing and simulating a large class of quantum error-correcting codes under Clifford operations. Shor's code was one of the first examples that motivated this powerful framework, which now underlies most practical quantum error correction schemes.*



4.3 ERROR CORRECTION

4.3.1 SYNDROME MEASUREMENT PROTOCOL

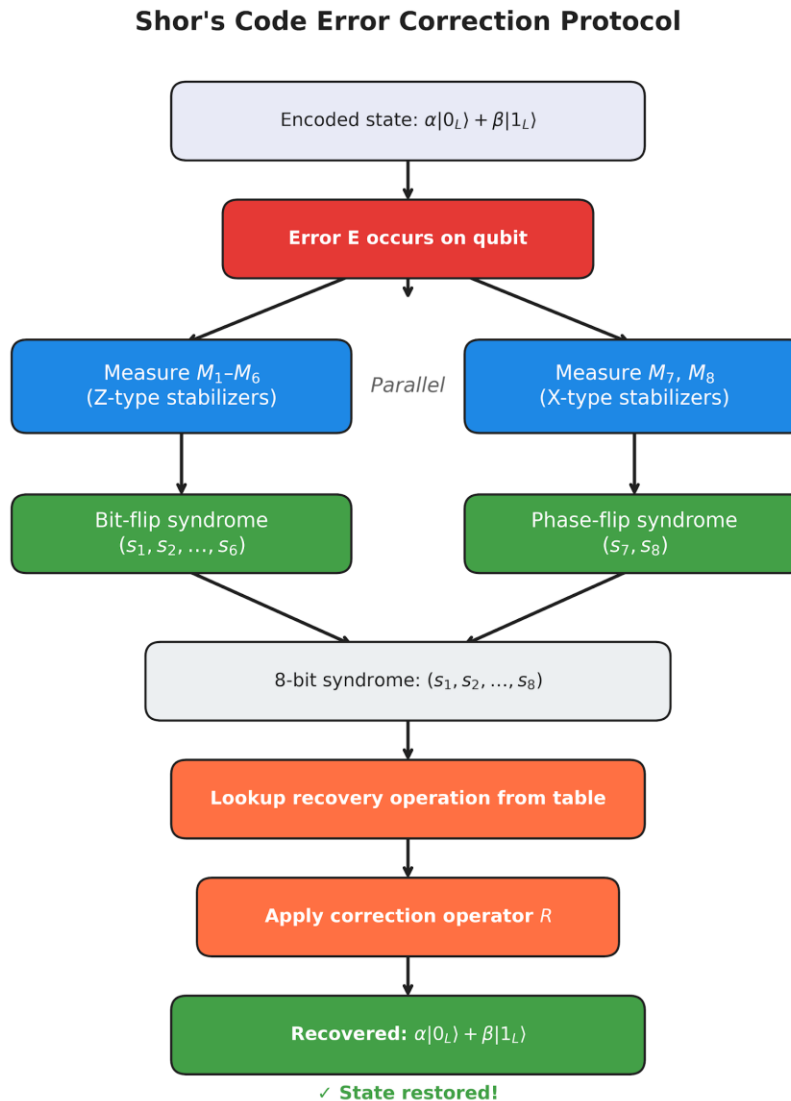


Figure 4.4: **Error Correction Protocol Flowchart (Conceptual Overview)** — from Encoded State through Syndrome Measurement to Recovery.



Error correction in Shor's code proceeds by measuring all eight stabilizer generators to extract a syndrome — an 8-bit string that identifies which error occurred without revealing any information about the encoded quantum state.

The measurement process:

1. **Measure each generator M_1 through M_8 :** Use ancilla-based circuits to determine each eigenvalue
2. **Record the syndrome:** Each measurement yields +1 or -1; record as $(s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$
3. **Decode the syndrome:** Use a lookup table to identify which error (if any) occurred
4. **Apply recovery:** Perform the Pauli operation that reverses the identified error

Non-destructive measurement: Crucially, measuring the stabilizer generators does not disturb the encoded quantum information. As Nielsen and Chuang explain, the measurement compares qubits pairwise to determine whether they agree or differ, without collapsing the logical state.

The measurement only reveals information about *errors*, not about the encoded logical state. This is because both $|0L\rangle$ and $|1L\rangle$ are +1 eigenstates of all generators — they are completely indistinguishable to stabilizer measurements. Whether the encoded state is $|0L\rangle$, $|1L\rangle$, or any superposition $\alpha|0L\rangle + \beta|1L\rangle$, the syndrome only depends on the error, not on α and β .

Ancilla-based measurement circuit for Z_1Z_2 :

To measure $M_1 = Z_1Z_2$ without directly measuring qubits 1 and 2:

5. Prepare an ancilla qubit in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$
6. Apply controlled-Z from the ancilla to qubit 1 (CZ gate)
7. Apply controlled-Z from the ancilla to qubit 2 (CZ gate)
8. Measure the ancilla in the X-basis (apply H, then measure in Z-basis)
9. Outcome $|+\rangle$ indicates eigenvalue +1; outcome $|-\rangle$ indicates eigenvalue -1



This circuit works because the controlled-Z operations accumulate a phase that depends on the parity of qubits 1 and 2. If both are $|0\rangle$ or both are $|1\rangle$ (even parity, eigenvalue +1), the ancilla remains in $|+\rangle$. If they differ (odd parity, eigenvalue -1), the ancilla flips to $|-\rangle$.

4.3.2 BIT-FLIP ERROR DETECTION AND CORRECTION

Bit-flip (X) errors within each block are detected by the Z-type generators M_1 through M_6 . Each pair of generators within a block creates a unique syndrome pattern for each possible single-qubit X error.

As Nielsen and Chuang describe, the Shor code can protect against both phase-flip and bit-flip errors on any qubit. Consider what happens when a bit-flip occurs on the first qubit.

Syndrome table for Block 1 (generators M_1, M_2):

Error	$M_1 (Z_1Z_2)$	$M_2 (Z_2Z_3)$	Syndrome	Recovery
None	+1	+1	(0,0)	None
X_1	-1	+1	(1,0)	Apply X_1
X_2	-1	-1	(1,1)	Apply X_2
X_3	+1	-1	(0,1)	Apply X_3

Table 4.8: Bit-flip Syndrome Table for Block 1 of the Shor Code — the Same Pattern Applies to Blocks 2 and 3 with Their Respective Generators.

How this works: The generator $M_1 = Z_1Z_2$ anticommutes with X_1 and X_2 (since $ZX = -XZ$ on the same qubit), but commutes with X_3 (since they act on different qubits). Similarly, $M_2 = Z_2Z_3$ anticommutes with X_2 and X_3 , but commutes with X_1 .

When an error E anticommutes with a generator M , the measurement outcome flips: if the original state had M -eigenvalue +1, the corrupted state has M -eigenvalue -1 . If E commutes with M , the eigenvalue is unchanged.



The same syndrome patterns apply to Blocks 2 and 3 using generators (M_3, M_4) and (M_5, M_6) respectively.

Worked example: X_2 error on qubit 2

1. **Initial state:** $\alpha|0L\rangle + \beta|1L\rangle$, which has eigenvalue +1 for all generators
2. **Error occurs:** X_2 is applied, giving $X_2(\alpha|0L\rangle + \beta|1L\rangle)$
3. **Measure $M_1 = Z_1Z_2$:** X_2 anticommutes with Z_2 , so the eigenvalue flips to -1
4. **Measure $M_2 = Z_2Z_3$:** X_2 anticommutes with Z_2 , so the eigenvalue flips to -1
5. **Measure M_3 through M_8 :** X_2 commutes with all of these (no overlap), eigenvalues remain +1
6. **Syndrome:** $(-1, -1, +1, +1, +1, +1, +1, +1)$, which uniquely identifies X_2
7. **Recovery:** Apply X_2 , which squares to identity: $X_2X_2 = I$, restoring the original state

4.3.3 PHASE-FLIP ERROR DETECTION AND CORRECTION

Phase-flip (Z) errors are detected by the X-type generators M_7 and M_8 . These generators compare the relative phases between adjacent blocks.

As Nielsen and Chuang explain, phase-flip errors are handled in a similar manner. A phase flip on any qubit in the first block flips the sign of that block, changing $(|000\rangle + |111\rangle)$ to $(|000\rangle - |111\rangle)$.

Key insight — degeneracy: A Z error on *any* qubit within a block produces the same effect on the block state. Consider Block 1:

- $Z_1(|000\rangle + |111\rangle) = Z_1|000\rangle + Z_1|111\rangle = |000\rangle + (-1)|111\rangle = |000\rangle - |111\rangle$
- $Z_2(|000\rangle + |111\rangle) = |000\rangle + (-1)|111\rangle = |000\rangle - |111\rangle$
- $Z_3(|000\rangle + |111\rangle) = |000\rangle + (-1)|111\rangle = |000\rangle - |111\rangle$

All three Z errors produce identical transformations on the block! The sign change affects only the $|111\rangle$ component, and since any single Z applied to $|111\rangle$ gives $-|111\rangle$, the three errors are indistinguishable at the block level.

This is why Shor's code is called a **degenerate code** — multiple distinct physical errors have the same effect on the code space and therefore the same syndrome.



Syndrome table for phase-flip errors (generators M_7, M_8):

Error Location	M_7	M_8	Syndrome	Recovery
None	+1	+1	(0,0)	None
Z on qubit 1, 2, or 3	-1	+1	(1,0)	$Z_1Z_2Z_3$
Z on qubit 4, 5, or 6	-1	-1	(1,1)	$Z_4Z_5Z_6$
Z on qubit 7, 8, or 9	+1	-1	(0,1)	$Z_7Z_8Z_9$

Table 4.9: Phase-flip Syndrome Table for the Shor Code — Z Errors on Any Qubit Within a Block Produce the Same Block-level Syndrome (degeneracy).

Why $Z_1Z_2Z_3$ for recovery, not just Z_1 ?

Since Z_1 , Z_2 , and Z_3 all produce the same syndrome, we cannot determine which specific qubit was affected. However, we don't need to — applying $Z_1Z_2Z_3$ correctly restores the block state regardless of which qubit had the original error.

To verify: $Z_1Z_2Z_3$ acting on the corrupted block state ($|000\rangle - |111\rangle$):

- $Z_1Z_2Z_3|000\rangle = (+1)(+1)(+1)|000\rangle = |000\rangle$
- $Z_1Z_2Z_3|111\rangle = (-1)(-1)(-1)|111\rangle = -|111\rangle$

Therefore $Z_1Z_2Z_3(|000\rangle - |111\rangle) = |000\rangle - (-1)|111\rangle = |000\rangle + |111\rangle \checkmark$

The recovery works because three Z operations on $|111\rangle$ give $(-1)^3 = -1$, flipping the sign back.

***Common Pitfall:** Students often try to apply Z to just one qubit when correcting phase-flip errors. While this would work if we knew which qubit was affected, we don't — the syndrome only tells us which block. The correct recovery $Z_1Z_2Z_3$ works regardless of which specific qubit had the original error, because all single-Z errors on a block are equivalent.*



4.3.4 COMBINED ERRORS AND ERROR DISCRETIZATION

Combined errors ($Y = iXZ$):

The Y Pauli error is equivalent to both a bit-flip and a phase-flip: $Y = iXZ$. When a Y error occurs, both the X and Z components are detected independently.

As Nielsen and Chuang note, when both bit and phase flip errors occur on a single qubit (i.e., the operator Z_1X_1 is applied), the bit-flip detection procedure identifies and corrects the bit-flip component, while the phase-flip detection procedure independently identifies and corrects the phase-flip component.

The key insight is that X and Z error corrections are *independent*:

- The bit-flip syndrome (from M_1 – M_6) identifies the X component $\rightarrow$ correct with the appropriate X
- The phase-flip syndrome (from M_7 – M_8) identifies the Z component $\rightarrow$ correct with $Z_1Z_2Z_3$ (or the appropriate block)
- The order of correction doesn't matter, since X and Z on the same qubit commute up to a phase

Complete syndrome table (selected entries):

Error	M_1	M_2	M_3	M_4	M_5	M_6	M_7	M_8	Recovery
None	+	+	+	+	+	+	+	+	None
X_1	–	+	+	+	+	+	+	+	X_1
X_5	+	+	–	–	+	+	+	+	X_5
Z (block 1)	+	+	+	+	+	+	–	+	$Z_1Z_2Z_3$
Z (block 2)	+	+	+	+	+	+	–	–	$Z_4Z_5Z_6$
$Y_1 = iX_1Z_1$	–	+	+	+	+	+	–	+	$X_1, Z_1Z_2Z_3$



$Y_5 =$									
iX_5Z_5	+	+	-	-	+	+	-	-	$X_5, Z_4Z_5Z_6$

Table 4.10: Selected Entries from the Full 8-generator Syndrome Table, Showing How Pure X, Pure Z, and Combined Y Errors Each Produce a Unique Syndrome Pattern.

The Discretization Principle:

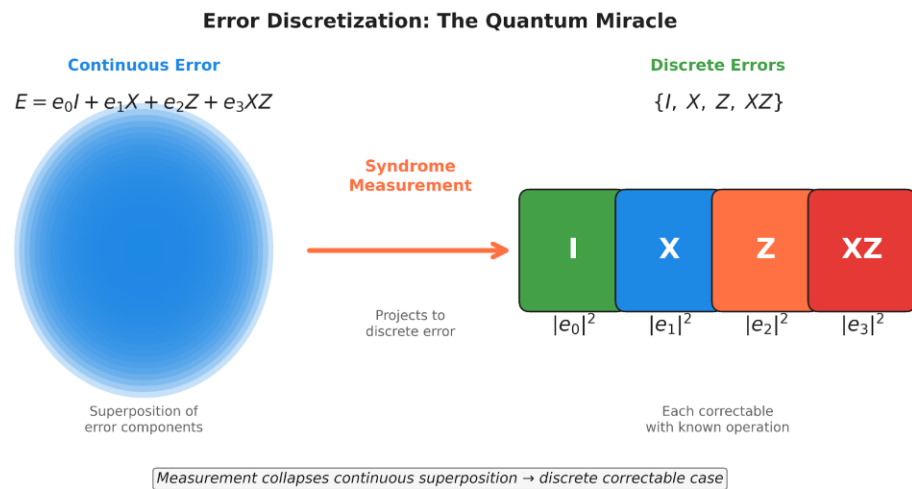


Figure 4.5: Error Discretization — Syndrome Measurement Collapses a Continuous Superposition of Errors into Discrete Correctable Cases.

Perhaps the most remarkable aspect of quantum error correction is that correcting a *finite* set of errors automatically corrects a *continuous* range of errors. This seems impossible at first — how can discrete corrections handle continuous noise?

Nielsen and Chuang emphasize: "This is an example of the extraordinary fact that the apparent *continuum* of errors that may occur on a single qubit can all be corrected by correcting only a *discrete* subset of those errors."

Any single-qubit error operator can be decomposed as:

$$E = e_0I + e_1X + e_2Z + e_3XZ$$



How to Read: "Any single-qubit error E can be written as a linear combination of the identity (no error), X (bit-flip), Z (phase-flip), and XZ (both), with complex coefficients e_0, e_1, e_2, e_3 ."

Where:

- E is an arbitrary single-qubit error operator (any 2×2 matrix)
- e_0, e_1, e_2, e_3 are complex numbers
- $\{I, X, Z, XZ\}$ form a complete basis for the space of 2×2 matrices
- For a unitary error preserving normalization: $|e_0|^2 + |e_1|^2 + |e_2|^2 + |e_3|^2 = 1$

Before syndrome measurement: The corrupted state is a superposition across error subspaces:

$$E|\psi_L\rangle = e_0|\psi_L\rangle + e_1X|\psi_L\rangle + e_2Z|\psi_L\rangle + e_3XZ|\psi_L\rangle$$

How to Read: "The error E acting on the encoded state ψ_L produces a superposition of four terms: the original state weighted by e_0 , plus the bit-flipped state weighted by e_1 , plus the phase-flipped state weighted by e_2 , plus the combined-error state weighted by e_3 ."

This is a continuous superposition — the error hasn't been "decided" yet.

After syndrome measurement: The measurement *projects* the state onto one of the error subspaces. The syndrome outcome is random:

- Probability of "no error" syndrome: $|e_0|^2$
- Probability of "X error" syndrome: $|e_1|^2$
- Probability of "Z error" syndrome: $|e_2|^2$
- Probability of "XZ error" syndrome: $|e_3|^2$

After projection, we know exactly which discrete error occurred and can apply the corresponding correction. The continuous superposition has collapsed to a definite error state.



As Nielsen and Chuang emphasise, error discretisation is central to why quantum error correction works, standing in stark contrast to classical error correction for analogue systems, where no such discretisation is possible.

This connects to the Knill-Laflamme conditions from Unit 3: the code satisfies these conditions for the error set $\{I, X, Z, XZ\}$, which guarantees that syndrome measurement projects errors cleanly and correction succeeds.

***Tips and Tricks:** The magic is in the measurement. Syndrome extraction doesn't just detect errors — it collapses continuous errors into discrete categories that we know how to correct. This is fundamentally quantum mechanical and has no classical analogue.*

***Activity:** Trace through the complete error correction protocol for a $Y_5 = iX_5Z_5$ error on qubit 5. Show:*

- (a) Which syndrome bits are affected by the X_5 component*
- (b) Which syndrome bits are affected by the Z_5 component*
- (c) The complete syndrome pattern*
- (d) The recovery operations needed and the order in which they can be applied*

4.4 SUMMARY

In this unit, you have learnt about Shor's $[[9,1,3]]$ code — the first quantum error-correcting code capable of protecting against arbitrary single-qubit errors:

- Shor's code uses **concatenation**, nesting the three-qubit bit-flip code inside the three-qubit phase-flip code to protect against both X and Z errors simultaneously.
- The code parameters $[[9,1,3]]$ mean nine physical qubits encode one logical qubit with distance 3, enabling correction of any single-qubit error.



- The logical codewords $|0L\rangle$ and $|1L\rangle$ are entangled superpositions of 8 computational basis states each, differing only in the relative signs within each block.
- The two-stage encoding circuit uses 8 CNOT gates and 3 Hadamard gates — Stage 1 creates phase-flip protection across blocks, and Stage 2 adds bit-flip protection within blocks.
- Eight stabiliser generators (six Z-type for bit-flip detection, two X-type for phase-flip detection) define the code space and enable syndrome measurement.
- Measuring all eight generators produces an 8-bit syndrome that uniquely identifies single-qubit Pauli errors without disturbing the encoded quantum information.
- Shor's code is **degenerate**: Z errors on any qubit within a block produce the same syndrome and are corrected by the same block-level operation.
- The **error discretisation** principle shows that correcting the discrete set $\{I, X, Z, XZ\}$ automatically corrects all continuous single-qubit errors — a fundamental insight with no classical analogue.

4.5 MIND MAP

[TO BE ADDED]

4.6 GLOSSARY

Block — A group of three physical qubits within Shor's code; the nine qubits form three blocks, each bit-flip encoded.

Codeword — An encoded logical state in the code space, such as $|0L\rangle$ or $|1L\rangle$ or any valid superposition.



Concatenation — Nesting one quantum code inside another to combine their error-correcting capabilities.

Degenerate code — A quantum code where multiple distinct physical errors produce the same syndrome and identical effect on the code space.

Error discretisation — Syndrome measurement projecting continuous errors into discrete correctable subspaces.

Logical qubit — Encoded quantum information protected by an error-correcting code using multiple physical qubits.

Stabiliser generator — A Pauli operator with eigenvalue +1 on all codewords; its measurement reveals error information without disturbing the encoded state.

Syndrome — The pattern of stabiliser measurement outcomes that identifies which error occurred; an 8-bit string for Shor's code.

4.7 SELF-ASSESSMENT QUESTIONS

1. Shor's code has distance $d = 3$, which means it can correct any single-qubit error. Why does the concatenation of two distance-1 codes (bit-flip and phase-flip) produce a distance-3 code rather than a distance-2 code?
 - a) Because $1 + 1 + 1 = 3$ from using three blocks
 - b) Because the two codes protect against complementary error types (X and Z), and any single-qubit error decomposes into at most one X and one Z component, both of which are independently correctable
 - c) Because the nine qubits provide enough redundancy to achieve distance 3
 - d) Because concatenation always doubles the distance and adds one
2. The three-qubit bit-flip code cannot detect phase-flip (Z) errors. What property of the Z operator explains why it is invisible to the Z_1Z_2 and Z_2Z_3 stabilizer measurements?
 - a) Z errors have no physical effect on qubits



- b) Z commutes with Z , so ZZ stabilizers cannot distinguish Z -corrupted states from uncorrupted states
 - c) Z errors only affect multi-qubit states
 - d) The bit-flip code does not use measurement
- 3. Why are Hadamard gates necessary in Stage 1 of Shor's encoding circuit?
 - a) They create entanglement between the three blocks
 - b) They convert the Z -basis entangled state to the X -basis, which is required for the bit-flip CNOTs in Stage 2 to produce the correct $(|000\rangle \pm |111\rangle)$ block states
 - c) They implement the phase-flip correction
 - d) They reduce the circuit depth from 12 to 11 gates
- 4. In Shor's code, a Z error on qubit 1, qubit 2, or qubit 3 all produce the same syndrome. What does this degeneracy imply about the recovery operation?
 - a) The error cannot be corrected because the specific qubit is unknown
 - b) A single correction operation $(Z_1 Z_2 Z_3)$ restores the block regardless of which qubit was affected, because all three errors have the same effect on the block state
 - c) Each qubit must be corrected individually in sequence
 - d) The code must be re-encoded after correction
- 5. Why does syndrome measurement in Shor's code not reveal the encoded quantum information α and β ?
 - a) The measurement apparatus cannot detect superpositions
 - b) Both logical codewords $|0_L\rangle$ and $|1_L\rangle$ are $+1$ eigenstates of all stabilizer generators, so the syndrome depends only on the error, not on which codeword was encoded
 - c) The ancilla qubits absorb the quantum information
 - d) The Knill-Laflamme conditions prevent information leakage by construction



6. Given syndrome $(-1, +1, +1, +1, +1, +1, +1, +1)$ for generators M_1 through M_8 , which error has occurred and what is the recovery operation?
 - a) X_2 error; apply X_2
 - b) X_3 error; apply X_3
 - c) X_1 error; apply X_1
 - d) Z_1 error; apply $Z_1Z_2Z_3$
7. An X_3 error occurs on qubit 3 of the encoded state. Which generators produce a -1 outcome?
 - a) $M_1 = Z_1Z_2$ only
 - b) $M_2 = Z_2Z_3$ only
 - c) Both M_1 and M_2
 - d) M_7 and M_8
8. A continuous rotation error $E = \cos(\theta/2)I - i\sin(\theta/2)Z$ acts on qubit 5 of Shor's code. What happens after syndrome measurement?
 - a) The error cannot be corrected because it is continuous
 - b) The state collapses to either "no error" (with probability $\cos^2(\theta/2)$) or "Z error on block 2" (with probability $\sin^2(\theta/2)$), and the appropriate correction is applied
 - c) The rotation is partially reversed by a proportional correction
 - d) Both I and Z corrections are applied simultaneously
9. The logical codeword $|1_L\rangle$ expanded as a sum of computational basis states has 8 terms. What determines the sign (+ or -) of each term?
 - a) The sign is always positive for $|1_L\rangle$
 - b) The sign is negative when an odd number of blocks are in the $|111\rangle$ state, positive when an even number are
 - c) The sign alternates between consecutive terms
 - d) The sign depends on the total number of 1s in the binary string



10. Shor's code has encoding rate $k/n = 1/9$, while the five-qubit code achieves $k/n = 1/5$.

Both correct arbitrary single-qubit errors. What advantage does Shor's concatenated structure offer despite its lower rate?

- a) It has a higher code distance
- b) It requires fewer measurement ancillas
- c) Its two-level structure (bit-flip inside phase-flip) makes syndrome extraction and analysis conceptually simpler, and it naturally extends to higher levels of concatenation
- d) It can correct two-qubit errors

11. The normalisation factor for the logical codewords of Shor's code is .

- a) $\sqrt{2}$
- b) $2\sqrt{2}$
- c) 4
- d) 8

12. The number of independent stabiliser generators needed to define the code space of Shor's $[[9,1,3]]$ code is .

- a) 6
- b) 8
- c) 9
- d) 3

13. True or False: A Z error on qubit 2 and a Z error on qubit 3 of Shor's code produce different syndromes.

- a) True, because they act on different physical qubits with distinct stabiliser interactions
- b) False, because both produce the same block-level phase flip in Block 1, yielding identical syndromes
- c) True, because generator $M_1 = Z_1Z_2$ distinguishes them



- d) False, because Z errors are undetectable by any stabiliser in Shor's code
14. True or False: The six CNOT gates in Stage 2 of the encoding circuit must be applied sequentially because they share target qubits.
- a) True, because sequential application prevents interference between blocks
 - b) False, because all six operate on different qubit pairs and can be applied in parallel
 - c) True, because the control qubits are entangled from Stage 1
 - d) False, because CNOT gates always commute regardless of qubit overlap

4.8 TERMINAL QUESTIONS

4.8.1 SHORT ANSWER QUESTIONS

1. Explain why Shor's code uses 9 qubits rather than 6 qubits. Why can't we simply run a 3-qubit bit-flip code and a 3-qubit phase-flip code in parallel on the same 3 qubits?
2. Compare the error-correcting capabilities of the three-qubit bit-flip code with Shor's nine-qubit code. What additional protection does the concatenated structure provide?
3. Explain the role of the three Hadamard gates in Stage 1 of the Shor code encoding circuit. What would happen to the encoding if they were omitted?
4. Why is Shor's code called a "degenerate" code? Illustrate your answer with an example involving phase-flip errors on Block 1.
5. Describe how the eight stabiliser generators of Shor's code divide into two groups. What type of error does each group detect, and how many generators are in each group?



4.8.2 LONG ANSWER QUESTIONS

1. Explain why the recovery operation for a detected phase-flip error on Block 1 is $Z_1Z_2Z_3$ rather than simply applying Z to the single affected qubit. Show the mathematical verification.
2. Derive the complete syndrome table for all nine single-qubit X errors (X_1 through X_9). Show the syndrome pattern for the Z -type generators M_1 – M_6 and explain how each error produces a unique pattern within its block.
3. Explain the error discretisation principle and discuss why it is often called "the miracle of quantum error correction." Include the mathematical decomposition of arbitrary errors in the Pauli basis and describe how syndrome measurement transforms a continuous error into discrete correctable cases.
4. Trace the complete encoding process for an arbitrary input state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ through both stages of the Shor code encoding circuit. Show the quantum state after each stage and verify that the final nine-qubit state matches the logical codeword definitions.
5. A $Y_3 = iX_3Z_3$ error occurs on qubit 3 of Shor's encoded state. Determine the complete 8-bit syndrome by analysing which generators anti-commute with each component, identify the error, and describe the full recovery procedure.

4.9 ANSWERS

4.9.1 SELF-ASSESSMENT ANSWERS

1. b) Because the two codes protect against complementary error types (X and Z), and any single-qubit error decomposes into at most one X and one Z component, both of which are independently correctable
2. b) Z commutes with Z , so ZZ stabilizers cannot distinguish Z -corrupted states from uncorrupted states



3. b) They convert the Z-basis entangled state to the X-basis, which is required for the bit-flip CNOTs in Stage 2 to produce the correct $(|000\rangle \pm |111\rangle)$ block states
4. b) A single correction operation $(Z_1 Z_2 Z_3)$ restores the block regardless of which qubit was affected, because all three errors have the same effect on the block state
5. b) Both logical codewords $|0L\rangle$ and $|1L\rangle$ are +1 eigenstates of all stabilizer generators, so the syndrome depends only on the error, not on which codeword was encoded
6. c) X_1 error; apply X_1
7. b) $M_2 = Z_2 Z_3$ only
8. b) The state collapses to either "no error" (with probability $\cos^2(\theta/2)$) or "Z error on block 2" (with probability $\sin^2(\theta/2)$), and the appropriate correction is applied
9. b) The sign is negative when an odd number of blocks are in the $|111\rangle$ state, positive when an even number are
10. c) Its two-level structure (bit-flip inside phase-flip) makes syndrome extraction and analysis conceptually simpler, and it naturally extends to higher levels of concatenation
11. b) $2V2$
12. b) 8
13. b) False, because both produce the same block-level phase flip in Block 1, yielding identical syndromes
14. b) False, because all six operate on different qubit pairs and can be applied in parallel

4.9.2 TERMINAL QUESTION ANSWERS

SHORT ANSWER QUESTIONS

1. Running both codes in parallel on the same three qubits fails because their protection mechanisms interfere — ZZ stabilisers commute with Z errors and XX stabilisers commute with X errors. Concatenation places them at different hierarchical levels, so both protections operate simultaneously without interference.



2. The three-qubit bit-flip code corrects only X errors; it is blind to Z errors since its ZZ stabilisers commute with Z. Shor's concatenated code corrects any single-qubit error — pure X, pure Z, combined Y, and continuous superpositions thereof.
3. The Hadamards convert the Z-basis entangled state $\alpha|000\rangle + \beta|111\rangle$ into $\alpha|+++ \rangle + \beta|--- \rangle$ on block leaders. Without them, Stage 2 CNOTs would produce a nine-qubit repetition code with no phase-flip protection.
4. Multiple distinct physical errors produce the same syndrome. For Block 1, Z_1 , Z_2 , and Z_3 all map $(|000\rangle + |111\rangle)$ to $(|000\rangle - |111\rangle)$, producing identical syndrome $(-1, +1)$ for M_7 , M_8 . The single recovery $Z_1Z_2Z_3$ corrects all three.
5. Six Z-type generators (M_1 – M_6) detect bit-flip errors using pairwise ZZ operators within each block. Two X-type generators (M_7 – M_8) detect phase-flip errors by comparing relative phases between adjacent blocks.

LONG ANSWER QUESTIONS

1. The syndrome identifies only the affected block, not the specific qubit, because Z_1 , Z_2 , and Z_3 produce identical block-state transformations. $Z_1Z_2Z_3$ acts on $(|000\rangle - |111\rangle)$ as: $Z_1Z_2Z_3|000\rangle = |000\rangle$ and $Z_1Z_2Z_3|111\rangle = (-1)^3|111\rangle = -|111\rangle$, restoring $(|000\rangle + |111\rangle)$. Applying Z to any single qubit also works because the residual is always a stabiliser element acting as identity on the code space. Refer to section 4.3.3 for detailed analysis.
2. Each block uses generators (Z_iZ_j , Z_jZ_k) producing three unique patterns: first qubit $\rightarrow (-1, +1)$, second $\rightarrow (-1, -1)$, third $\rightarrow (+1, -1)$. For Block 1 (M_1, M_2): $X_1 \rightarrow (-1, +1)$, $X_2 \rightarrow (-1, -1)$, $X_3 \rightarrow (+1, -1)$. Blocks 2 and 3 follow the same pattern with their respective generators (M_3, M_4) and (M_5, M_6). The full 6-bit syndrome uniquely identifies the affected qubit since only one block's generators produce non-trivial outcomes. Refer to Table 4.8 for the complete syndrome table.
3. Any error E decomposes as $E = e_0I + e_1X + e_2Z + e_3XZ$. Before measurement, the corrupted state is a continuous superposition across error subspaces. Syndrome



measurement projects this onto one discrete subspace — "no error" with probability $|e_0|^2$, "X error" with $|e_1|^2$, "Z error" with $|e_2|^2$, or "XZ error" with $|e_3|^2$. After projection, standard Pauli correction restores the state. This works because the Knill-Laflamme conditions guarantee orthogonality between error subspaces. Refer to section 4.3.4 for the full derivation.

4. Input: $\alpha|0\rangle + \beta|1\rangle$ on qubit 1, $|0\rangle$ on qubits 2–9. Stage 1: CNOTs entangle block leaders to $\alpha|000\rangle + \beta|111\rangle$; Hadamards convert to $\alpha|+++ \rangle + \beta|--- \rangle$. Stage 2: CNOT pairs expand each $|\pm\rangle$ to $(|000\rangle \pm |111\rangle)/\sqrt{2}$. Final state: $\alpha(|000\rangle + |111\rangle) \otimes^3 / 2\sqrt{2} + \beta(|000\rangle - |111\rangle) \otimes^3 / 2\sqrt{2} = \alpha|0L\rangle + \beta|1L\rangle$. Refer to sections 4.2.2 and 4.2.3 for gate-by-gate details.
5. X_3 anti-commutes with Z_3 in $M_2 = Z_2Z_3$ (giving -1) but commutes with $M_1 = Z_1Z_2$ ($+1$). M_3 – M_6 give $+1$. Z_3 anti-commutes with X_3 in M_7 (giving -1) but commutes with M_8 ($+1$). Complete syndrome: $(+1, -1, +1, +1, +1, +1, -1, +1)$. Recovery: apply X_3 for the bit-flip, then $Z_1Z_2Z_3$ for the block-level phase-flip. Refer to section 4.3.4 for combined error correction details.

4.10 REFERENCES

BOOKS

- Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th Anniversary Edition). Cambridge University Press.
- Gottesman, D. (1997). *Stabilizer Codes and Quantum Error Correction*. PhD Thesis, California Institute of Technology.

JOURNAL ARTICLES



- Shor, P. W. (1995). Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4), R2493–R2496. <https://doi.org/10.1103/PhysRevA.52.R2493>
- Brun, T. A. (2019). Quantum error correction. *arXiv preprint*, arXiv:1910.03672. <https://arxiv.org/abs/1910.03672>

WEB REFERENCES

- Devitt, S. J., Munro, W. J., & Nemoto, K. (2013). *Quantum Error Correction for Beginners*. <https://arxiv.org/abs/0905.2794>
- Quantum Error Correction, Wikipedia. <https://en.wikipedia.org/wiki/Quantumerrorcorrection>

The mathematical development of Shor's $[[9,1,3]]$ code is correct. However, additional quantum circuit diagrams—particularly for encoding, stabiliser measurement, and GHZ block preparation—are recommended to accurately represent syndrome extraction and error discretisation processes in this mathematically intensive unit.

Minor mathematical notation refinements (stabiliser commutation, logical state representation, and tensor-product equivalence) and figure-caption clarification have been incorporated. No conceptual modifications were required.